Shadow API Discovery & Vulnerability Scanner

A rule-based security scanner for detecting undocumented API endpoints and confirming Broken Object Level Authorization vulnerabilities

---

HCIC-SI2026 — Problem 15 Submission

Author / Lead: Ekansh Jaiswal

Date: July 2026

Status: Working build, actively improved

Repository: github.com/ekansh-jaiswal/ShadowAPI-Scanner

Live Demo: Runs locally as a command-line tool (no hosted demo)

This project does not use machine learning. Every finding comes from a plain, checkable rule, and where possible, the tool confirms the finding with a real network request instead of just guessing.

1 Overview

Objective: Build a security scanner, with no machine learning involved, that finds "Shadow APIs" (parts of a system that quietly receive traffic but were never documented), and then checks whether they're actually exploitable.

The Challenge: Digital health systems, like India's ABDM-style setups, expose a lot of web APIs. Over time, some of these get forgotten: an old debug route nobody removed, a private link used by some internal dashboard, a "temporary" endpoint that ended up in production and stayed there. These are dangerous exactly because nobody remembers they exist. If nobody knows about it, nobody is checking it, even if it's still handling sensitive health data.

What Exactly Is a Shadow API? (And What's a Zombie API?) These two terms get used loosely, so it's worth being precise, since this project is specifically about the first one. A **Shadow API** is an endpoint that was never officially documented or approved in the first place — a developer builds something quickly to hit a deadline, it goes live, and it never gets added to the official spec or reviewed by a security team. Nobody did anything malicious. It just slipped in under the radar and stays invisible to normal monitoring, because as far as any official record is concerned, it doesn't exist. A **Zombie API** is a different problem with a similar result: an endpoint that *was* once official and documented, but was never properly shut down when it should have been — like an old API version nobody decommissioned after an upgrade. It just keeps running, forgotten, still reachable, still carrying whatever weaknesses it had when it was retired, except now nobody is watching it either. The difference matters for detection: Shadow APIs need to be *discovered* (traffic going somewhere the documentation doesn't mention), while Zombie APIs need to be *tracked over time* (something that used to matter has quietly gone stale while still technically alive). This project focuses on Shadow APIs specifically. The Temporal Drift Detection idea listed later, under What's Next, is the natural extension toward catching Zombie APIs too.

The Solution: A Python command-line tool that: (1) reads web server logs to see every endpoint that's actually getting hit, (2) compares that against an OpenAPI spec to find anything not officially listed, (3) runs seven checks based on the OWASP API Security Top 10 (2023) against each one it finds, and (4) for endpoints that hold a specific user's data, sends a real request trying to access another user's data with it. That last part is the main point: the tool doesn't guess that something might be broken. It tries to break it, for real, and shows you exactly what happened.

Does It Actually Work?: Tested against a fake health gateway ("SwasthyaConnect") built with five endpoints deliberately left undocumented, the scanner found all five, with no wrong guesses, and proved three serious Broken Object Level Authorization (BOLA) bugs live. Each one is backed by a real captured request and response, showing one patient's Aadhaar number, diagnosis, and prescription being returned using a different patient's login. Just as important, the tool correctly left two normal, safe endpoints alone (a public health check and a public doctor listing) that an earlier version of the code had wrongly flagged. A scanner nobody trusts because it keeps crying wolf isn't much better than no scanner at all, so cutting down wrong guesses mattered as much as catching the real problems.

How to Try It: This is a command-line tool, not a hosted website, so there's no live link to click. Anyone can run it themselves by cloning the repository and following the setup steps in the README — it takes a few minutes and includes a ready-to-use fake environment for testing, so no separate account or server needs to be set up first.

2 How It's Built

Backend / Scripting: Python 3.11, Flask (used to run a fake API server for testing), Jinja2 (used to build the report file)

Data & Parsing: PyYAML to read the OpenAPI spec, openapi-spec-validator to check that spec is actually valid before trusting it, and a regex-based reader for Nginx-style access logs

Report Output: A single HTML file with everything built in. No external scripts, no internet connection needed to open it and click around — it works the same on any machine, offline.

System Flow:

```
[Access Logs + Auth-Header Log] + [OpenAPI Spec]
→ [Log Parser / Spec Loader]
→ [Diff Engine:  Shadow / Dormant / Documented]
→ [Risk Engine:  7 OWASP checks + live BOLA probe]
→ [Scorer:  weighted 0--100 risk score per endpoint]
→ [HTML Report Generator]
→ [CLI: exit codes ready for CI/CD]
```

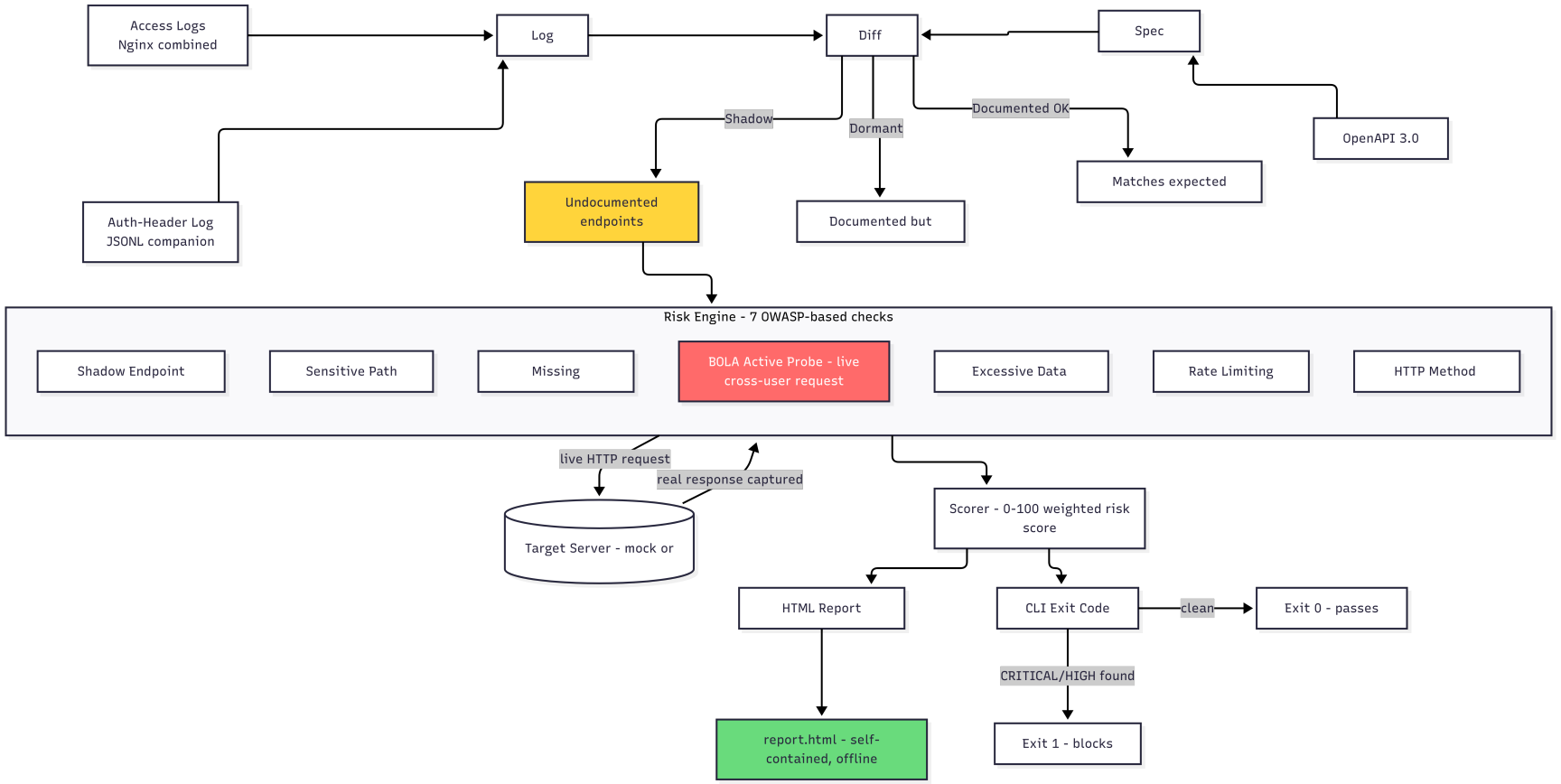

Figure 1: Shadow API Scanner — how data flows through the system

3 What It Actually Does

Shadow API Discovery: Compares real traffic against an OpenAPI spec to find endpoints that were never documented. URLs like `/patient/104` and `/patient/117` are correctly matched as the same endpoint, `/patient/{id}`, instead of being counted as two separate ones just because the numbers differ.

Live BOLA Check: The scanner doesn't just guess that an endpoint might be broken based on traffic patterns. It sends a real request using one user's login token to try to reach another user's data, and records exactly what comes back. A finding is only marked CRITICAL if this actually works — the report shows the real evidence, not a prediction.

What "Rule-Based" Actually Means: Each check is a plain, human-written condition, not a trained model guessing at patterns. For example, the missing-authentication check is simply: "does the OpenAPI spec say this endpoint needs a login, and did at least one real request in the logs reach it without one?" If yes, flag it. There's no probability score, no training data, no black box — every single finding can be traced back to one exact rule, and that rule can be read directly in the code. This matters for a security tool specifically because a security team needs to know exactly why something was flagged, not just that a model was "pretty confident."

Why the Spec Has to Come From Somewhere Else: A fair question is why the scanner needs a human, or a separate system, to hand it an OpenAPI spec in the first place — why doesn't it just generate its own spec by watching traffic? The answer is that doing so would defeat the entire purpose of the tool. If the spec were built from the same traffic being audited, every endpoint the scanner ever saw would automatically "match the spec," since the spec would just be a description of what already happened. Nothing could ever be flagged as undocumented. The whole point depends on the spec being an independent source of truth — a record of what was *supposed* to exist, kept separate from what's *actually* happening — so that a mismatch between the two is a meaningful finding rather than a tautology.

In a real deployment, this independent source is usually easy to get without any manual work: most API gateways and frameworks (Kong, AWS API Gateway, FastAPI, Spring with springdoc, and others) can export their own registered OpenAPI spec automatically, since they already track which routes they officially expose. Pointing the scanner's `--spec` flag at that export is normally a one-line setup step, not a manual writing exercise.

For the OWASP crAPI validation described in Section 6, this automatic export wasn't available, since crAPI is an intentionally under-documented teaching application and does not publish a machine-readable spec of its own. A partial spec was hand-written instead, based only on the endpoints a reasonable API consumer would expect to find officially documented. This is not a shortcut around the tool's design — it mirrors the exact real-world situation the tool is built for: an organization whose internal documentation is incomplete, which is itself one of the most common root causes of Shadow APIs existing in the first place.

How It Decides What's What, Step by Step: The process runs in four stages, and every stage is a plain yes/no or a fixed formula — nothing in here is a guess.

Stage 1 — Is it in the spec? For every endpoint the logs show real traffic for, the tool checks one thing: does this exact path (after normalizing IDs, so `/patient/104` and `/patient/117` count as the same path) appear in the OpenAPI spec? If yes, it's labeled Documented OK. If no, it's labeled Shadow, and moves to Stage 2. This is a plain set comparison — nothing fuzzy about it.

Stage 2 — Run the 7 checks. Every Shadow endpoint gets checked against all 7 rules: does the path look like it holds sensitive data (keywords like "patient" or "aadhaar" in the URL)? Does it ever get hit with no login at all? Does a real cross-user request actually succeed (the BOLA probe)? Does the response leak fields it shouldn't (SSNs, diagnosis codes)? Does it get hammered

with no rate limit? Is it being called with a method the spec never listed? Each check either fires or it doesn't — there's no partial match, no confidence percentage.

Stage 3 — Add up the weight. Each check that fires adds a fixed number of points based on its severity: CRITICAL adds 40, HIGH adds 25, MEDIUM adds 10, LOW adds 5, INFO adds 1. These get summed per endpoint and capped at 100 if they go over, which is common, since a real BOLA finding usually triggers several checks together.

Stage 4 — Turn the score into an action. The final 0–100 score gets bucketed into a severity band — CRITICAL, HIGH, MEDIUM, LOW — and that decides the color on the report and, more importantly, whether the tool blocks a deployment. If the person running the scan set `--fail-on critical,high` (the default), any endpoint landing in the CRITICAL or HIGH band makes the whole scan exit with a failure code, which is exactly what a CI/CD pipeline would check before letting new code go live.

The whole point of laying it out this way is that every single number in the final report can be traced backward through these four stages to one specific rule. If a finding looks wrong, you can always ask which of the 7 checks fired, and why, and get a real, exact answer, not a confidence score.

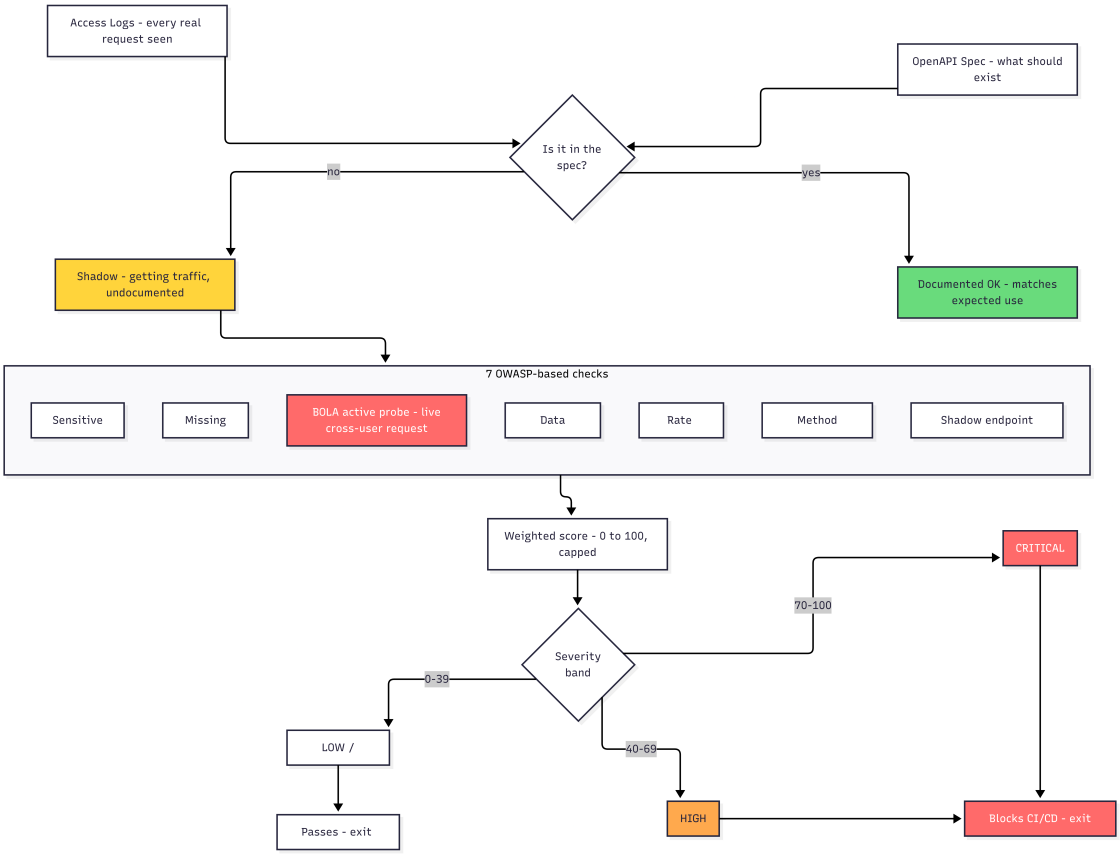

Figure 2: How a raw request becomes a labeled endpoint, then a severity score

Avoiding False Alarms: An earlier version of this check fired on any endpoint with a number in the URL, which meant it wrongly flagged a public doctor listing as broken — looking up any doctor by ID is completely normal, there's no "owner" of a doctor's public profile to protect. This was caught while testing by hand and fixed by only running the cross-user check on endpoints that

actually belong to a specific person's data. That removed this whole category of wrong result.

DPDP Act 2023 Notes: Findings involving personal or health data get a short note pointing to relevant sections of India's DPDP Act — Section 6 on limiting what data is collected and why, Section 8(1) on keeping data reasonably secure, and so on. These notes are written as "relevant to" or "may be worth reviewing," never "this breaks the law." The tool points out technical facts. It isn't a lawyer and doesn't try to make legal calls.

4 Building It and Testing It

About the Data Used Here: All the demo data — patient records, Aadhaar-shaped numbers, diagnoses — is made up. No real personal data and no real systems were touched at any point while building or testing this. The live-probing feature is meant only for systems the user owns or has clear permission to test.

Handling Failure Properly: While testing by hand, a real bug turned up: running the scanner with the test server turned off still produced the exact same "confirmed" CRITICAL results as when the server was on. That's a serious problem for a tool whose whole point is "we checked this for real, we didn't just guess." The cause was an error being silently ignored inside the probe code. The fix adds a quick check at startup: if the target server can't be reached, the scanner shows a clear warning, both on screen and in the report, falls back to results based on logs alone, and the risk score drops to match — from 100 (CRITICAL) down to 61 (HIGH) in testing — instead of quietly repeating old results.

Testing: Every part of the pipeline was checked against real output, not assumed to work. This included trying to break it on purpose — missing files, a broken spec file, turning the server off mid-scan — not just testing the case where everything goes right. The exit-code behavior, which is what a real deployment pipeline would use to stop a bad release, was checked across six different scenarios. This testing caught a real bug where one setting (`--fail-on medium`) failed to catch a CRITICAL result, because the code was checking for an exact match instead of checking "is this at least as serious as medium."

5 The Hard Part, and What's Left to Do

What Made This Hard: Telling a real authorization bug apart from a normal public lookup, without using machine learning to do it. A simple rule like "test any endpoint with a number in the URL" ends up flagging things that were never meant to belong to any one person, like a public doctor listing.

How It Was Fixed: Added a step that checks whether an endpoint's name suggests it holds a specific person's data — using words like patient, appointment, insurance, otp, debug, prescription — so the cross-user check only runs where it actually makes sense to run it. Confirmed this worked by re-running the scan and checking that the doctor listing and health-check endpoints came back clean, while the three real bugs still got caught.

What's Next:

- A graph view showing how discovered endpoints relate to each other — shared login patterns, grouped by resource type
- Comparing two sets of logs taken at different times, to catch newly appeared shadow endpoints early

- Moving past keyword matching for the ownership check, toward actually comparing responses across different user tokens, so it still works on real APIs that don't use the same naming style as this project's test setup

6 External Validation via OWASP crAPI

Why: Test the scanner against a well-known, independently built vulnerable application — OWASP's Completely Ridiculous API (crAPI) — instead of only the custom test environment built specifically for this project. A tool that only finds bugs in its own test fixture doesn't prove much on its own. Finding bugs in something it wasn't built around is a stronger, more convincing test.

What Was Done: Two separate user accounts were created through crAPI's real signup flow, each verified by email through crAPI's own MailHog inbox, and each user added a vehicle using the VIN and PIN sent to them. Real traffic was captured automatically using `mitmproxy` configured as a reverse proxy in front of crAPI, recording every request made during normal use — no requests were hand-picked. This capture was mechanically converted into the Nginx-format log the scanner expects, and a partial OpenAPI spec was written covering only the endpoints a reasonable consumer would expect to find officially documented (see Section 3 for why the spec has to come from an independent source, not from the traffic itself).

A Generalization Fix Along the Way: The first automated run against crAPI correctly flagged `/identity/api/v2/vehicle/{id}/location` as a Shadow endpoint, but the active BOLA probe did not fire on it. The cause was a real limitation: the probe's ownership-scope check used a small, hardcoded keyword list built around this project's own healthcare test data, so it had no way to recognize a vehicle-management application's vocabulary. Rather than simply adding "vehicle" to the list, which would only have fixed this one case, the check was rebuilt to detect ownership scoping structurally — any path containing an instance identifier (a numeric ID or a UUID) is treated as a candidate for the BOLA probe by default, with a short, explicitly documented exclusion list for known public reference data (doctor directories, product catalogs, health checks, and the like). This makes the detection logic generalize to applications the tool was never tuned for, instead of requiring manual keyword updates for every new target.

Result — Confirmed Live, Fully Automated: After the fix, the scanner's own CLI — using only the vehicle UUIDs it extracted itself from the captured log, with no IDs supplied by hand — ran the BOLA probe and confirmed the vulnerability live. Using User B's real login token (`userb@shadowscan.test`), the scanner successfully requested User A's vehicle location at `/identity/api/v2/vehicle/fadcf145-85a5-4967-9691-44f75706a026/location` and received an HTTP 200 response containing User A's real GPS coordinates, full name, and email address. This matches OWASP's own officially documented crAPI Challenge 1, confirming both that this is a known, real vulnerability class, and that the scanner's detection logic — not just a manually crafted request — independently finds it.

Regression Check: The same fix was re-run against this project's own mock environment to confirm nothing broke. `/api/v1/doctors/{id}` and `/api/v1/health` still correctly produced zero findings, and all three original BOLA vulnerabilities (`patient-records`, `internal/debug/patient`, `insurance-claims`) still triggered correctly, unchanged.

A Second Bug, Found While Confirming the First Fix: Re-running the full CLI against crAPI initially produced a confusing result: the risk engine was correctly finding and returning a CRITICAL-severity BOLA finding (verified directly by calling the risk-engine function in isolation), but the endpoint's overall label in the report and terminal summary showed HIGH, not CRITICAL.

The cause turned out to be in the scorer, not the probe. `determine_risk_level(score)` buckets an endpoint purely by its summed weighted score (CRITICAL findings add 40 points, HIGH add 25, MEDIUM add 10, and so on, capped at 100), using fixed thresholds to assign the final band. An endpoint carrying one CRITICAL finding (40) alongside an unrelated MEDIUM finding (10, in this case the routine "Shadow API Endpoint" flag every undocumented path gets) summed to 50, which lands in the 50–74 HIGH band — silently under-labeling a confirmed, evidence-backed CRITICAL vulnerability as HIGH, both in the report's severity badge and in the CLI's `--fail-on` exit message. The underlying finding was never lost or wrong; it was just diluted by an unrelated lower-severity finding on the same endpoint when the two scores were summed and bucketed together.

The fix is narrow: an endpoint's final risk level is now the more severe of (a) the existing summed-score band, and (b) the single highest individual finding severity present on that endpoint. The 0–100 score itself is unchanged and still reflects the summed weight, since it remains useful for ranking endpoints relative to each other — only the severity label and the pass/fail decision now respect a CRITICAL finding as a floor. After the fix, re-running against crAPI shows `/identity/api/v2/vehicle/{id}/location` correctly labeled CRITICAL (score unchanged at 50/100), with the terminal summary reading `Overall Risk Exposure: CRITICAL` and the exit message correctly reporting CRITICAL findings present. The same re-run against the mock environment confirmed no regression: all three original CRITICAL endpoints remained unchanged at 100/CRITICAL, `/doctors/{id}` and `/health` remained clean, and one additional endpoint, `/api/v1/patients/{id}`, was correctly promoted from HIGH (66) to CRITICAL (66) — verified by inspecting its finding cards directly rather than trusting the score alone, confirming a genuine, evidence-backed BOLA finding (a real HTTP 200 response returning another patient's data via a different user's token) that had been sitting under this same masking bug the whole time.

What's Next:

- Extend the exclusion-list override to a config file, so a user can add application-specific public-reference terms without editing source code, beyond the `--exclude-from-bola` flag added during this fix
- Explore whether the same structural approach (ID-in-path detection) can be extended to catch ownership-scoped resources that don't follow a clean REST `/resource/{id}` pattern
- Apply the same "highest individual finding as a floor" principle more broadly during scoring review, in case other check combinations produce similar dilution on endpoints with several findings of mixed severity

7 How to Use This Tool

Prerequisites: Python 3.11 or newer, and a virtual environment. From the repository root:

```
python3.11 -m venv .
source bin/activate
pip install -r requirements.txt
```

If `pip install` complains about an externally-managed environment instead of installing cleanly, that means the venv wasn't actually activated first — check `which python` and `which pip` both point inside the venv's `bin/` directory before retrying. This came up directly while testing this project: running `pip install pyyaml` outside an activated venv threw an externally-managed-environment error, and the fix was to activate the venv properly rather than force the install with `--break-system-packages`, which would silently install into the system Python instead.

Two Ways to Run This: **(a) Against the included mock environment** — a safe, fully self-contained fake health-gateway API (`mock_env/`) built specifically so anyone can try the tool in minutes with no setup risk and no real data involved. This is the right starting point for a first run. **(b) Against a real target** — pointing the scanner at real access logs and a real OpenAPI spec. **Only do this against systems you own or have explicit written permission to test.** The active BOLA probe makes real authenticated network requests attempting cross-user access — running it against a system you don't control or haven't been authorized to test is unauthorized access, full stop, regardless of intent.

Walkthrough: Mock Environment

This needs two terminals open at once, because the mock server is a blocking process that has to keep running while the scanner (in a separate terminal) makes requests to it.

Terminal 1 — start the mock server, leave it running:

```
cd project
source bin/activate
python mock_env/mock_server.py
```

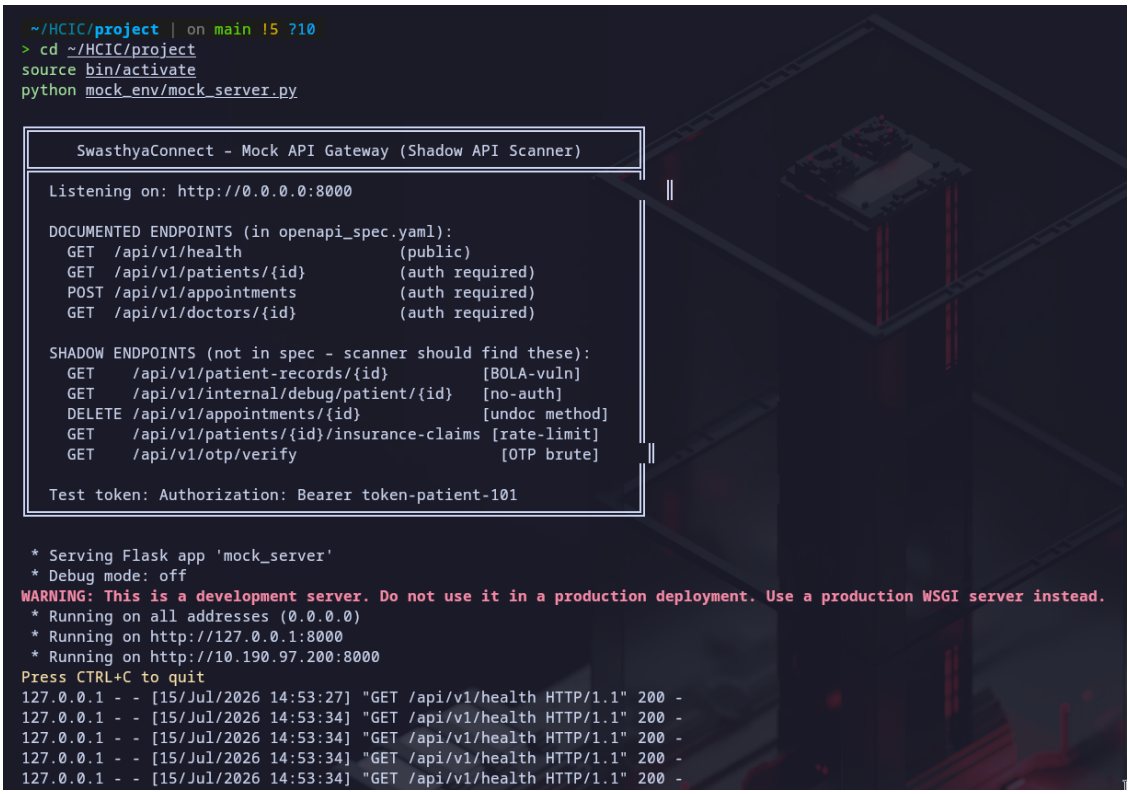

Figure 3: Mock server startup: documented vs. shadow endpoints listed at launch

Don't touch this terminal again until you're done. It listens on port 8000 until you press Ctrl+C.

Terminal 2 — generate traffic, then run the scan:

```
cd project
source bin/activate
```

```
python mock_env/generate_logs.py
wc -l mock_env/access.log mock_env/access_headers.log
python scanner/cli.py \
  --log-file mock_env/access.log \
  --spec mock_env/openapi_spec.yaml \
  --mock-server-url http://localhost:8000 \
  --output report.html \
  --fail-on critical,high
echo "Exit code: $?"
```

Figure 4: Traffic generation: 485 log entries written, requests by normalised endpoint

Figure 5: Traffic breakdown by method, status code, and source IP, followed by scanner startup

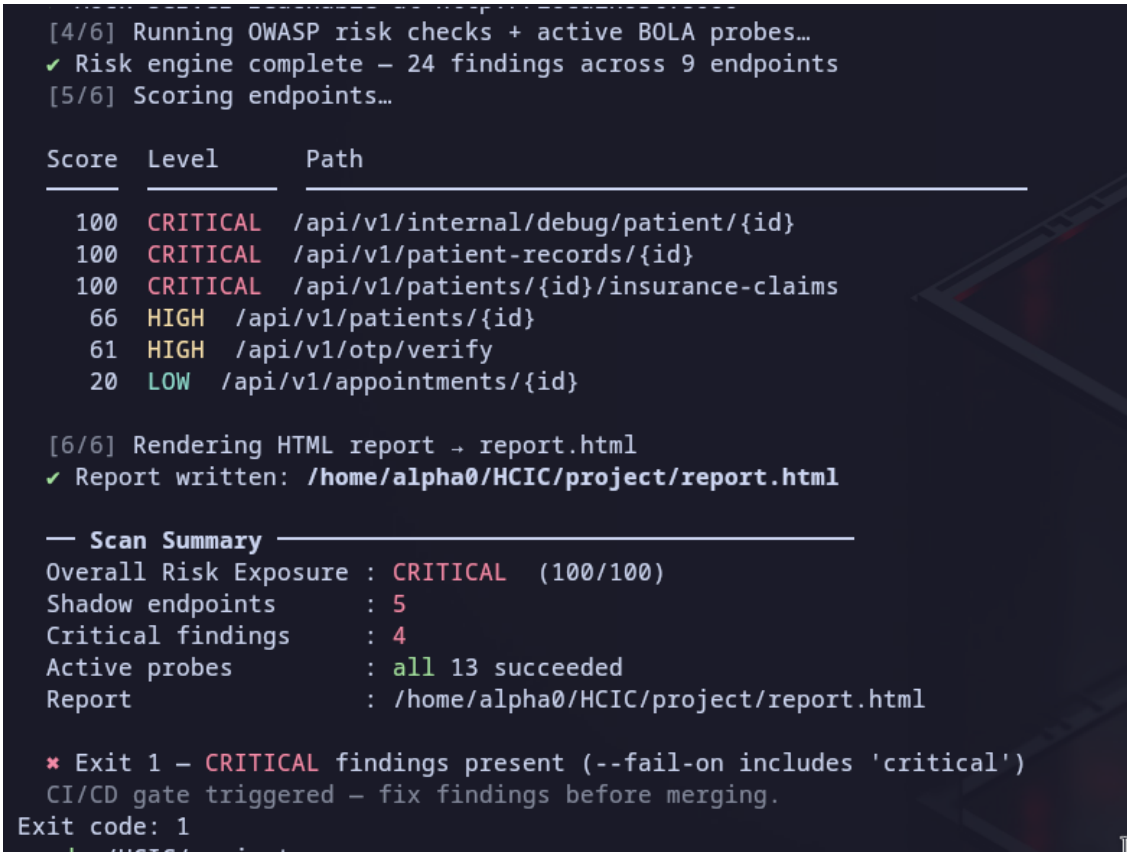

Figure 6: Endpoint scoring table and final scan summary with exit code

What Each Flag Does:

- `--log-file` — path to the Nginx-format access log to read real traffic from. Required.
- `--spec` — path to the OpenAPI spec describing what's officially documented. Required.
- `--mock-server-url` — base URL the active BOLA probe sends real requests to. If this is omitted, the scanner still runs, but skips live probing and reports passive findings only — functionally identical to passing `--no-active-probes`.
- `--no-active-probes` — explicitly disables live network requests. Use this when you want a fast, read-only pass over the logs and spec with no traffic sent anywhere, or when you don't have permission to make live requests against the target yet.
- `--output` — where to write the HTML report. Defaults to `report.html` if omitted.
- `--fail-on` — comma-separated severity floor (e.g. `critical,high`, or a single level like `medium`) that determines the process exit code. Any finding at or above the lowest level named makes the scan exit non-zero. This is the flag a CI/CD pipeline actually checks.
- `--quiet` — suppresses the stage-by-stage console output, printing only the final summary and exit code. Useful when the scanner is called from another script and only the exit code matters.
- `--version` — prints the tool's version and exits immediately, without needing `--log-file` or `--spec` at all.

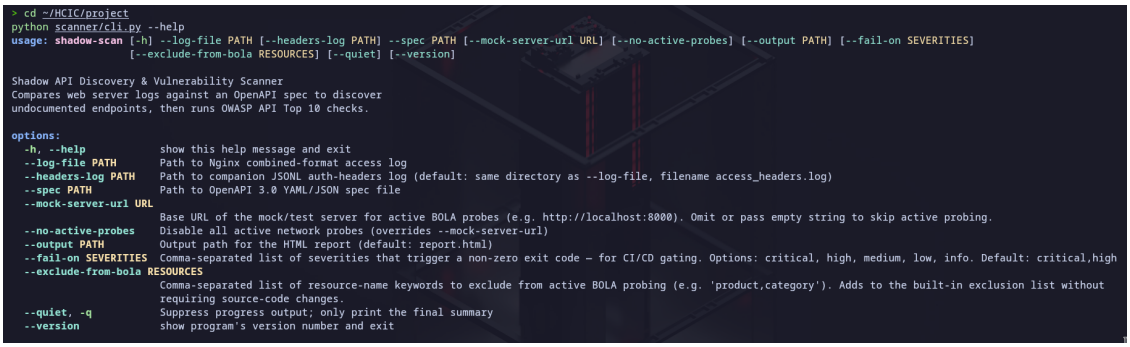

Figure 7: Full CLI help output, listing every flag and its default

Real Terminal Output From a Full Scan:

```
Log file : mock_env/access.log
Spec     : mock_env/openapi_spec.yaml
Headers  : mock_env/access_headers.log
Probes   : http://localhost:8000
Output   : report.html
Fail on  : CRITICAL, HIGH

[1/6] Parsing access logs...
  Parsed 485 log lines -> 9 unique path templates
[2/6] Loading OpenAPI spec...
  Loaded 'SwasthyaConnect API Gateway' v1.0.0 -- 4 documented paths
[3/6] Computing shadow/documented diff...
  Diff complete -- 5 shadow, 0 dormant, 4 documented OK
[4/6] Running OWASP risk checks + active BOLA probes...
  Risk engine complete -- 24 findings across 9 endpoints
[5/6] Scoring endpoints...
  Score  Level      Path
  -----  ---------  ------------------------------------------
  100    CRITICAL   /api/v1/internal/debug/patient/{id}
  100    CRITICAL   /api/v1/patient-records/{id}
  100    CRITICAL   /api/v1/patients/{id}/insurance-claims
  66     HIGH       /api/v1/patients/{id}
  61     HIGH       /api/v1/otp/verify
  20     LOW        /api/v1/appointments/{id}
[6/6] Rendering HTML report -> report.html
  Report written: /home/alpha0/HCIC/project/report.html

-- Scan Summary ------------------------------------------
Overall Risk Exposure : CRITICAL (100/100)
Shadow endpoints      : 5
Critical findings     : 4
Report                : /home/alpha0/HCIC/project/report.html

Exit 1 -- CRITICAL findings present (--fail-on includes 'critical')
```

```
CI/CD gate triggered -- fix findings before merging.
```

Reading the 6 Stages: Each numbered stage does exactly one job and prints one confirming line before moving to the next — there's no hidden work happening between stages. Stage 4 is the one doing the actual security analysis (the 7 OWASP checks plus the live BOLA probe); everything before it is just getting the data into a comparable shape, and everything after it is turning that analysis into a score and a report.

The Exit Code: This is what makes the tool usable in an automated pipeline, not just as a manual report generator. Exit `0` means no finding reached the `--fail-on` threshold — safe to proceed. Exit `1` means a finding at or above the threshold was found — a CI/CD job should treat this as a failed check and block the deploy. Exit `2` means a usage error, like a missing `--log-file` or `--spec` path — this is a configuration problem, not a security finding, and is handled separately so a pipeline doesn't confuse "we couldn't run the scan" with "the scan found something bad."

Pointing the Scanner at a Real System

Getting a real access log: Both Nginx and Apache can be configured to write access logs in the combined log format the scanner's parser expects. If a server isn't already logging in this format, most deployments can enable it with a one-line config change (Nginx: the `log_format combined` directive; Apache: `LogFormat "%h %l %u %t \"%r\" %>s %b" combined`) rather than needing custom logging code written from scratch.

Getting a real OpenAPI spec: As explained in Section 3 ("Why the Spec Has to Come From Somewhere Else"), the spec must come from an independent source of truth, separate from the traffic being audited — otherwise nothing could ever be flagged as undocumented, since the spec would just describe whatever already happened. In most real deployments this is not manual work: API gateways and frameworks that already track their own registered routes can export a spec directly, including Kong, AWS API Gateway, FastAPI, and Spring with springdoc. Pointing `--spec` at that export is normally a one-line setup step. Only when no such export exists — as with OWASP crAPI in Section 6, an intentionally under-documented teaching application — does the spec need to be hand-written, and even then it should only describe what a reasonable API consumer would expect to be documented, not be reverse-engineered from the same traffic being scanned.

The `--exclude-from-bola` flag: This flag exists because of a real generalization limit found during the crAPI validation (Section 6): the BOLA probe's ownership-scope check now treats any path containing an instance identifier (a numeric ID or a UUID) as a candidate for active probing by default, with a short built-in exclusion list for well-known public reference data (doctor directories, product catalogs, health checks). Real applications will have their own public, non-ownership-scoped lookups that aren't on that default list. Rather than editing the scanner's source code every time, pass a comma-separated list of resource names to skip:

```
python scanner/cli.py \
  --log-file access.log \
  --spec openapi_spec.yaml \
  --mock-server-url https://api.example.com \
  --exclude-from-bola branch,region,warehouse \
  --output report.html
```

This adds the given terms to the exclusion list at runtime, for that run only — it does not modify the built-in defaults.

Reading `report.html`

Executive Summary: The top of the report shows the overall risk exposure score (0–100, with a severity badge), the count of shadow endpoints found, and the count of critical findings — the same numbers printed in the terminal's Scan Summary, just rendered visually.

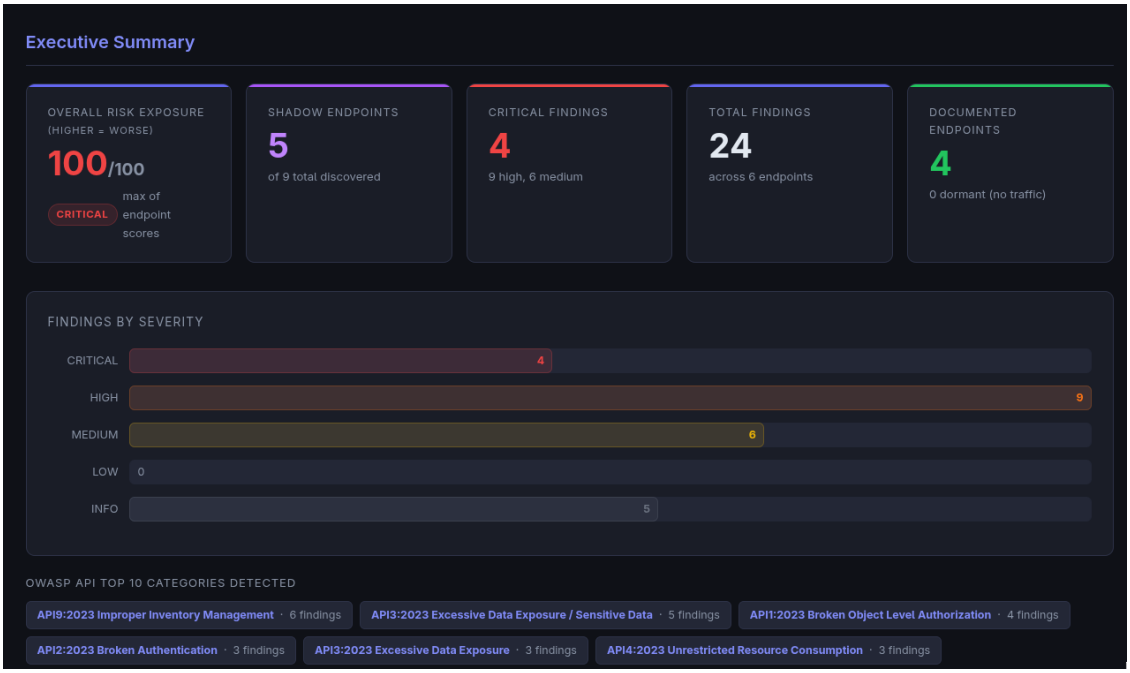

Figure 8: report.html Executive Summary: risk exposure, findings by severity, OWASP categories detected

Severity Badges: Every finding and every row in the attack-surface table carries a color-coded badge (CRITICAL, HIGH, MEDIUM, LOW, INFO) next to its numeric score, not just the bare number — so a reader scanning quickly still sees the severity at a glance without doing the score-to-band math themselves.

Attack-Surface Table: Lists every discovered endpoint with its path, HTTP method, shadow/documented status, and score. Columns are sortable and the severity badges are filterable, so a reviewer can, for example, click CRITICAL to see only the endpoints that matter most right now.

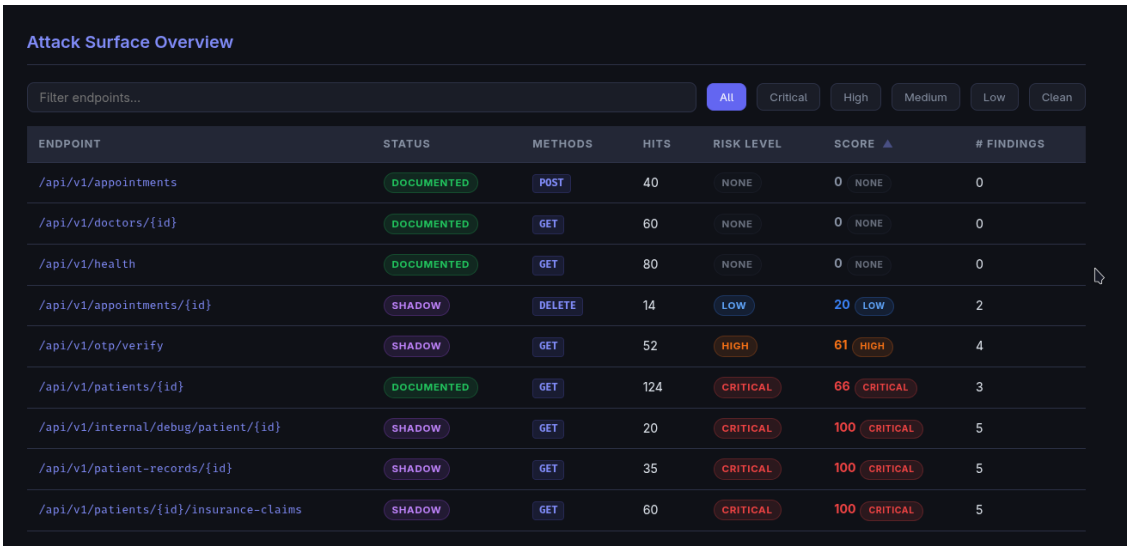

Figure 9: report.html Attack Surface Overview: sortable, filterable endpoint table

Finding Cards: Each endpoint expands into a card showing exactly which of the 7 checks fired and why. For a confirmed BOLA finding, the card includes the actual evidence block — the real request that was sent (which token, which target ID) and the real response that came back, not a description of what probably happened. This is deliberate: every CRITICAL finding should be traceable back to a specific piece of real evidence, not an inference.

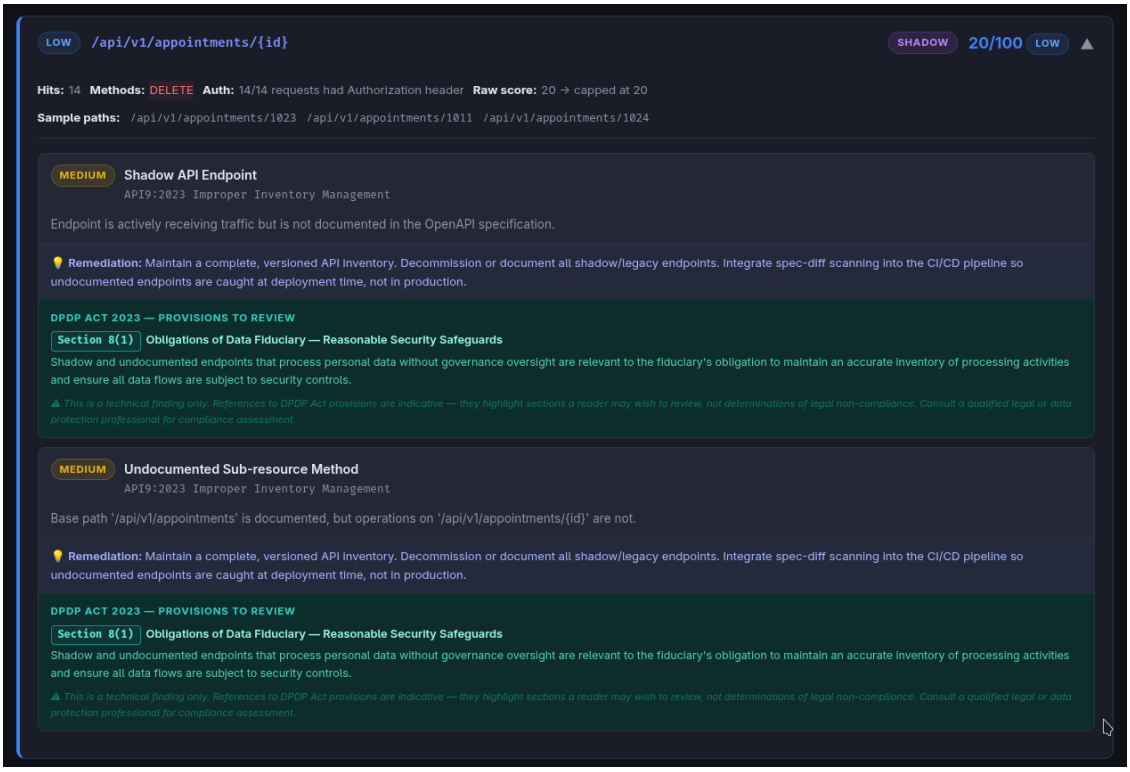

Figure 10: Expanded finding card: individual checks, OWASP category tags, and remediation guidance

DPDP Callouts: Findings touching personal or health data carry a short note referencing relevant sections of India's DPDP Act, 2023 (Section 6 on purpose limitation, Section 8(1) on reasonable security, and similar). These are deliberately worded as "relevant to" or "may be worth reviewing," never "this violates the law" — the tool surfaces technical facts, not legal conclusions.

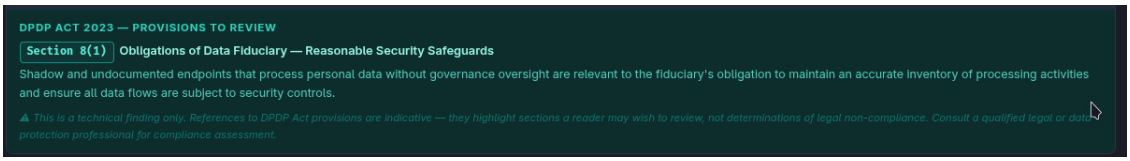

Figure 11: DPDP Act 2023 callout referencing Section 8(1), worded as a note for review rather than a legal conclusion

The Warning Banner: If the active probes couldn't reach the target when the scan ran, a prominent amber banner appears near the top of the report, above the Executive Summary, distinct in color from the red/orange severity badges so it's never mistaken for a finding. The real banner text, confirmed in testing with the mock server deliberately turned off:

```
Active Probes Skipped --- mock server unreachable at http://localhost:8000/api/v1/health.
Reason:  Connection refused or host unreachable (ConnectionError).  Re-run with a
live server or use --no-active-probes for passive-only mode.
```

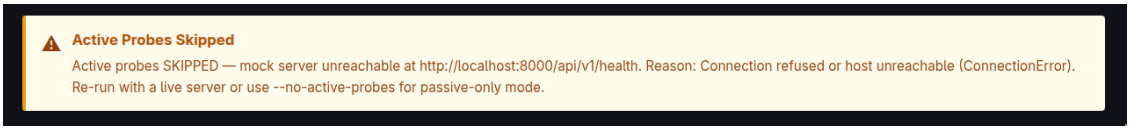

Figure 12: Warning banner shown when the active BOLA probe cannot reach the target server

When this banner is present, BOLA findings are not re-verified for that run — the report falls back to passive, log-based findings only, and the overall risk score reflects that (in testing, the same scan that scored CRITICAL/100 with the server up scored HIGH/61 with the server down, since the unconfirmed CRITICAL BOLA findings were correctly withheld rather than shown as if they'd been proven).

Troubleshooting

"Connection refused" when running the scanner against the mock server: The mock server isn't running, or it's running in the same terminal the scanner is being run from. `mock_server.py` is a blocking process — it needs its own terminal, left alone, for as long as the scanner or the log generator needs to reach it. Open a second terminal for everything else.

`LogMismatchError`: Raised when `access.log` and `access_headers.log` have a different number of lines — the parser refuses to guess which header line belongs to which request rather than silently misattributing auth data. The real error message looks like this:

```
access.log has 3 lines, access_headers.log has 2 lines -- they MUST match.
access_headers.log is 1 line(s) shorter than access.log.
Both files agree up to line 2; divergence starts at line 3.
Fix: re-run python mock_env/generate_logs.py to regenerate both files,
or call parse_logs(..., headers_log_path=None) to skip auth merging.
```

The fix is exactly what the message says: re-run `generate_logs.py` so both files are written together again in lockstep, rather than editing one file by hand.

Docker/containerd not running (only relevant if validating against crAPI): If `docker compose ps` fails with something like `failed to connect to the docker API at unix:///var/run/docker.so ... no such file or directory`, Docker's daemon isn't running. On Linux, check containerd first, since Docker depends on it directly:

```
sudo systemctl status containerd
sudo systemctl start containerd
sudo systemctl reset-failed docker.service
sudo systemctl start docker
sudo systemctl status docker
```

In this project's own testing, `containerd` had silently stopped after an earlier ungraceful shutdown, which caused Docker to fail immediately on start without logging any error of its own — so an empty `journalctl` grep for `dockerd` errors is itself a signal to check `containerd`'s status specifically, rather than continuing to retry `docker start`.

`pip install` failing with an externally-managed-environment error: This means the venv isn't actually activated, even if it looks like it from the prompt. Confirm with `which python` and `which pip` — both should point inside the project's `bin/` directory. If they point somewhere else (like `/usr/bin/python`), run `source bin/activate` again before installing. Reaching for `--break-system-packages` instead of fixing the activation papers over the problem and risks installing dependencies into the system Python rather than the project's isolated environment.

Sources

1. OWASP API Security Top 10 (2023) — https://owasp.org/API-Security/editions/2023/en/0x11-t10/
2. OpenAPI Specification v3.0.3 — https://spec.openapis.org/oas/v3.0.3
3. Nginx HTTP Log Module documentation — https://nginx.org/en/docs/http/ngx_http_log_module.html
4. Flask documentation — https://flask.palletsprojects.com/
5. Digital Personal Data Protection Act, 2023, Government of India — official gazette text
6. OWASP crAPI (Completely Ridiculous API) — https://github.com/OWASP/crAPI
7. openapi-spec-validator (PyPI) — https://pypi.org/project/openapi-spec-validator/
8. OWASP API Top 10 - Broken Authentication — https://youtu.be/ycAC24R8N4o?si=7HuMUcncCXQ9-1d8
9. Hacking APIs: Fuzzing 101 — https://youtu.be/M_guA0wjrLg?si=lIdq8XH2ylhJpr3q
10. Understanding Broken Object Level Authorization (BOLA) — https://medium.com/@godwincherry02/understanding-broken-object-level-authorization-bola-3a7a0ee1d516
11. Broken Object Level Authorization (BOLA) Explained — https://youtu.be/YciLnEY1AN0?si=FyS4HwkxwkSFp7WA
12. Hacking Your First API! — https://youtu.be/lYUYtBzrkDU?si=ovF_BTb372Sd8QWP
13. What are Shadow and Zombie APIs? Identifying API Security Risks — https://youtu.be/ewi6pMP11YY?si=_AV-obwgcGOQsSTq
14. Finding & Exploiting Mass Assignment Vulnerabilities — https://youtu.be/8-Q0nT3dv48?si=apTc6mMLTPBicW_w

15. AUTOMATING API Security TESTING (Ft. APISec Scanner) — https://youtu.be/LtBofiwDzb0?si=NL7zpUDXLNnc-_Uw
16. Manage Mock Rules Via APIs — https://youtu.be/NsfJ_wrl4Kc?si=oSSipM7hidQL9IGi
17. Top 12 Tips For API Security — https://youtu.be/6WZ6S-qmtqY?si=cC3ONydJBbfDuwgH
18. RICO Demo: AI-Powered API Security Scanner | OpenAPI Vulnerability Detection & CI/CD Protection — https://youtu.be/oBHD8bPp3dw?si=qQhWqIe8IQxjkwxB
19. Security Rules - CompTIA Network+ N10-009 - 4.3 — https://youtu.be/ORRcLSS9_Ps?si=i01DtV3uc4aw24SD
20. OWASP API Security Top 10 Course — Secure Your Web Apps — https://youtu.be/YYe0FdfdgDU?si=iMn7ZDlt5Gu1mJn
21. API Security Explained: Rate Limiting, CORS, SQL Injection, CSRF, XSS & More — https://youtu.be/FsB_nRGdeLs?si=Yu4Y3hQhSJnuzDhk
22. OWASP API Top 10 Breakdown | Study Session with CTF Challenges (DVAPI) — https://youtu.be/oIwiGiMe82U?si=UG77JfbPkPE8wry2
23. API Security Fundamentals — Course for Beginners — https://youtu.be/R-4_DbV1Su4?si=MACAJeZr6ABKYVww
24. API Security - Top 7 Best Practises — https://youtu.be/W5IRONNlFrI?si=imsMRufr3szwEo3n
25. This CTF Teaches You Everything About Hacking an API — https://youtu.be/6Tyqvl-GSNQ?si=S6OiznhPcDnLjrQZ